

# Procesadores de Lenguajes Snapi

Pablo Peña Lozano, Adrian Carlos Skaczylo

## Índice

|                                                   |          |
|---------------------------------------------------|----------|
| <b>1. Identificadores y ámbitos de definición</b> | <b>2</b> |
| 1.1. Comentarios . . . . .                        | 2        |
| 1.2. Declaración de variables . . . . .           | 2        |
| 1.3. Declaración de arrays . . . . .              | 2        |
| 1.4. Bloques anidados . . . . .                   | 3        |
| 1.5. Funciones y paso de parámetros . . . . .     | 3        |
| <b>2. Tipos</b>                                   | <b>4</b> |
| 2.1. Tipos Básicos . . . . .                      | 4        |
| <b>3. Conjunto de instrucciones del lenguaje</b>  | <b>5</b> |
| 3.1. Asignación . . . . .                         | 5        |
| 3.2. Condicionales . . . . .                      | 5        |
| 3.3. Bucles . . . . .                             | 5        |
| 3.3.1. Bucle <code>while</code> . . . . .         | 5        |
| 3.4. Lectura y escritura . . . . .                | 6        |
| 3.5. Expresiones . . . . .                        | 6        |
| <b>4. Gestión de errores</b>                      | <b>6</b> |

# 1. Identificadores y ámbitos de definición

## 1.1. Comentarios

El lenguaje permitirá añadir comentarios de distintos tipos: comentarios de una línea y comentarios de líneas arbitrarias. Los comentarios del primer tipo se marcarán con el carácter `#`, mientras que los del otro tipo se delimitan mediante `/* ... */`.

```
1 # Comentario de una línea
2
3
4 /*
5 Comentario
6 de varias
7 líneas
8 */
```

## 1.2. Declaración de variables

El lenguaje permitirá declaración de variables y de arrays de cualquier tipo. La declaración de variables se indicarán con la palabra reservada **var**. El tipo de la variable aparece en primer lugar, seguido del símbolo `:` como separador y, a continuación, el identificador de la variable.

```
1 var tipo : identificador;
2
3 #Ejemplo
4 var int : contador;
```

También se permitirá asignar un valor directamente a una variable en el momento de su declaración, siendo esta una funcionalidad opcional.

```
1 var tipo : identificador = valor;
2
3 #Ejemplo
4 var int : contador = 1;
```

## 1.3. Declaración de arrays

El lenguaje permitirá declarar arrays de cualquier tipo válido, en la declaración se permite añadir varias dimensiones para crear arrays multidimensionales. La sintaxis general sigue el mismo patrón que las variables simples, usando la palabra reservada **var**, pero para indicar que se trata de un array se utiliza la palabra reservada **array** seguida de: corchetes con el tamaño; y el tipo, después de la palabra reservada **of**.

```
1 var array[size]...[size] of tipo : identificador;
2
3 #Ejemplo
4 var array[10] of int : lista;
5 var array[3][2] of bool : lista2;
```

## 1.4. Bloques anidados

El lenguaje permitirá el uso de bloques anidados, es decir, bloques de código que se encuentran dentro de otros bloques. Cada bloque se delimita mediante llaves `{ }`, y puede contener declaraciones de variables, instrucciones y otros bloques internos. No se permite, sin embargo, la declaración de funciones dentro de bloques, estas habrán de hacerse a nivel global.

```
1 {
2     var x : int;  # x existe solo en este bloque
3
4     {
5         var y : bool;
6         var z : int;
7     }
8
9     # Aqui solo se puede usar x, y no existe la y bool ni z
10 }
```

Cada bloque maneja su propia tabla de símbolos, lo que facilita la gestión de ámbitos y la detección de errores de declaración.

## 1.5. Funciones y paso de parámetros

El lenguaje permitirá definir funciones con paso de parámetros por valor y por referencia. Cada función se declara usando la palabra reservada **func**, seguida del tipo, el nombre de la función y los parámetros entre paréntesis. Se reservará la palabra **return** para indicar el resultado que devuelve la función.

```
1 func tipo : nombre(var tipo1 : param1, var tipo2 : param2,...){
2     #Cuerpo de la funcion
3 }
4
5
6 #Ejemplo
7
8 func int : suma( var int : x, var int: y){
9
10     var int : resultado;
11     resultado = x + y;
12
13     return resultado;
14 }
```

Los parámetros se pasan por valor por defecto, y se pasan por referencia utilizando el símbolo `&` antes del identificador del parámetro. Se podrán combinar ambos tipos de parámetros.

```
1 func tipo : nombre(var tipo1 : &param1, var tipo2 : param2,...){
2     #Cuerpo de la funcion
3 }
4
5
```

```

6 #Ejemplo
7
8 func int : suma( var int : &x, var int: y){
9
10     var int : resultado;
11     resultado = x + y;
12     x = x +1;
13
14     return resultado;
15 }

```

Los procedimientos tendrán por tipo la palabra reservada **void** y seguirán la misma estructura que las funciones:

```

1 func void : nombre(var tipo1 : &param1, var tipo2 : param2,...){
2     #Cuerpo del procedimiento
3 }
4
5
6 #Ejemplo
7 func void : swap( var int : &x, var int: &y){
8
9     var int : z = x;
10    x = y;
11    y = z;
12 }

```

## 2. Tipos

### 2.1. Tipos Básicos

El lenguaje implementa un sistema de tipos estático y explícito, en el que cada variable debe declararse con su tipo correspondiente. Los tipos básicos predefinidos son:

- **int** : enteros
- **bool** : booleanos

Se soportan operadores infijos sobre estos tipos, con prioridades y asociatividades definidas:

| Operador               | Tipo de operandos | Asociatividad |
|------------------------|-------------------|---------------|
| *, /                   | int, int          | izquierda     |
| +, -                   | int, int          | izquierda     |
| <, >, ==               | int, int          | no asociativo |
| <b>and</b> , <b>or</b> | bool, bool        | izquierda     |

## 3. Conjunto de instrucciones del lenguaje

### 3.1. Asignación

El lenguaje permite realizar asignaciones a variables simples y a posiciones específicas de arrays. La sintaxis general es:

```
1 identificador = expresion;  
2 identificador[indice] = expresion; # para arrays  
3  
4 #Ejemplo  
5 var int : x;  
6 var array[5] of int : numeros;  
7  
8 x = 10; # asignacion a variable simple  
9 numeros[2] = 7; # asignacion a un elemento del array
```

### 3.2. Condicionales

El lenguaje soporta condicionales con una o dos ramas, utilizando la palabra reservada **if** y **else**. La sintaxis general es:

```
1 if(condicion){  
2     # instrucciones si la condicion es verdadera  
3 }  
4 else {  
5     # instrucciones si la condicion es falsa (opcional)  
6 }  
7  
8 #Ejemplo  
9  
10 var int : x = 5;  
11  
12 if (x > 0) {  
13     print(x);  
14 } else {  
15     print(0);  
16 }
```

### 3.3. Bucles

El lenguaje permitirá el uso de estructuras de repeticion mediante bucles **while**. Este ejecuta un bloque de sentencias delimitado por llaves { }.

#### 3.3.1. Bucle while

La sintaxis del bucle **while** es la siguiente:

```
1 while(condicion){  
2     # Bloque de sentencias  
3 }
```

```

4
5 #Ejemplo
6
7 var int : i = 0;
8
9 while (i < 5) {
10     print(i);
11     i = i + 1;
12 }

```

### 3.4. Lectura y escritura

El lenguaje incluirá instrucciones para lectura y escritura de enteros:

```

1 read(identificador);      # lee un valor desde la entrada
2 print(expresion);         # imprime un valor
3
4 #Ejemplo
5 var int : edad;
6 read(edad);
7 print(edad);

```

### 3.5. Expresiones

Las expresiones pueden estar formadas por:

- Constantes y literales
- Identificadores simples o con subíndices (arrays)
- Operadores infijos con prioridades definidas
- Llamadas a funciones

```

1 var int : a = 5;
2 var int : b = 3;
3 var array[5] of int : numeros;
4
5 numeros[0] = a + b * 2;      # expresion con operadores infijos
6 var int : resultado = sumar(a, &b); # llamada a funcion

```

## 4. Gestión de errores

En caso de producirse un error durante la compilación, se indicará el tipo de error detectado junto con la fila y la columna en la que se ha producido. Tras detectar un error, el proceso de compilación no se detendrá inmediatamente, sino que se aplicarán mecanismos básicos de recuperación con el objetivo de continuar el análisis y detectar el mayor número posible de errores en una misma ejecución. Una vez finalizado el análisis, si se ha producido algún error, la compilación se considerará fallida.